

MADRE

Agentic
System

• MODEL-AGNOSTIC • DELAYED • REASONING • EFFORT • [Beta v0.1]

Safety And Authority Model

Scvria Software

June 20, 2026

Contents

Safety And Evolution Policy	1
1.1 Runtime Safety	1
1.1.1 Policy Before Action	1
1.1.2 Autonomy Levels	1
1.2 Evolution Control	2
1.2.1 Evolution Rules	2
Runtime Governance	3
2.1 Runtime Authority Rule	3
2.2 Policy And Action Gate	3
2.3 Policy Matrix	3
2.4 Learning Lifecycle	4
Threat Model	5
3.1 Threat Taxonomy	5
Threat Catalogue	7

Safety And Evolution Policy

1.1 Runtime Safety

MADRE is intended to run useful delayed work while the user is not constantly present. That requirement makes safety an architectural property, not a documentation afterthought. The system must be able to explain what it did, why it did it, what data it used, what boundary allowed it, and how the result can be corrected or rolled back when the action is reversible.

1.1.1 Policy Before Action

The kernel must not expose executable behavior merely because a model can request it. Internal actions, external calls, knowledge-boundary changes, workflow execution, and learning promotion need an explicit `PolicyDecision` before execution. The safe path should be the normal path, and safe autonomous improvement should not require constant user interruption. Unsafe paths should be technically isolated, traceable, recoverable where possible, and visible to the responsible developer or operator through `RuntimeJournal`.

Typed runtime-defined internal actions represent the canonical bounded action form. They may be allowed only through a policy-gated action boundary. Remote action, host commands, arbitrary filesystem access, mutation actions, provider-backed integrations, active learning promotion, and workflow mutation require stronger authority boundaries than bounded internal action execution.

1.1.2 Autonomy Levels

Level	Meaning
Observation	Read local runtime state, existing knowledge, and safe metadata without mutation.
Draft creation	Produce summaries, candidate records, or proposed changes without promoting them into active state.
Bounded mutation	Modify authorized local state only when the action is logged, scoped, reversible where possible, and policy-permitted.
External action	Contact remote systems, use external services, or disclose data only after classification, minimization, authorization, provenance, and recorded trace.
Critical action	Credentials, identity, legal, financial, destructive, irreversible, or executable-code actions require a stronger human authorization path.

1.2 Evolution Control

1.2.1 Evolution Rules

- Runtime learning is represented by evaluated **LearningCandidate** proposals, not only by manual feedback collection.
- Learning strategies must declare what they are allowed to observe, propose, mutate, verify, and promote.
- A **LearningCandidate** cannot silently modify protected behavior.
- Learning proposals begin with source, evaluation evidence, scope, and rollback information before they are promoted.
- A **KnowledgeCandidate** is used only for pending knowledge-boundary change and requires provenance, truth metadata, correction paths, and recovery options.
- Saving a **KnowledgeRecord** is not learning by itself.
- Workflow and routing improvements may become default behavior through developer-defined checks, evaluation gates, and rollback policies, without requiring user authorization for every routine improvement.
- Model adaptation, fine-tuning, or weight updates are advanced model-evolution actions governed by stronger model-adaptation authority.
- Critical learning failures must be recoverable through snapshots, versioned state, VCS-backed records, or an equivalent rollback mechanism.

This policy keeps **MADRE** capable of evolving through conscious, checked learning without reducing improvement to constant user authorization. The final user should be interrupted as little as possible. Routine learning should be governed by developer-defined strategies, evidence, tests, promotion gates, **RuntimeJournal** traces, and rollback mechanisms; user authorization is reserved for boundaries where risk, privacy, identity, external action, or policy explicitly requires it.

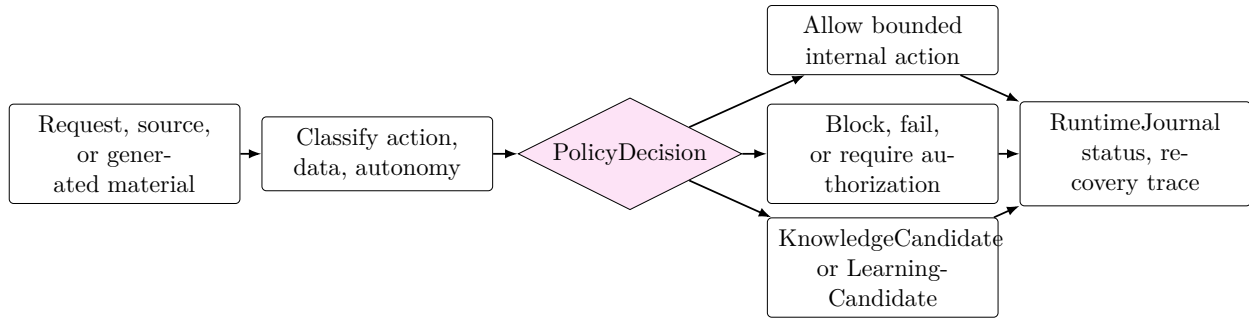
D-011 fixes the controlled-promotion rule: **KnowledgeCandidate** denotes pending knowledge-boundary change and **LearningCandidate** denotes evaluated proposed module improvement. Neither candidate creation nor **KnowledgeRecord** storage silently rewrites protected behavior.

Runtime Governance

2.1 Runtime Authority Rule

Runtime authority belongs to the **MADRE** kernel and its policy boundaries. User turns, source text, generated material, action results, local inspection, **KnowledgeCandidate**, and **LearningCandidate** can inform decisions, but they cannot authorize their own authority, promote themselves, mutate protected state, or widen permissions. Bounded internal action use requires **PolicyDecision**, local access is read-first where it inspects state, and transitions are recorded through **RuntimeJournal**.

2.2 Policy And Action Gate



2.3 Policy Matrix

Action class			Data class		Autonomy level		Authorization	Trace/Recovery
Read	local	meta-data	Public	or low-sensitivity metadata	low-local	Observation	None by default	Log access when used for reasoning.
Draft text			Any	allowed text bundle	con-	Draft creation	None unless sensitive scope is crossed	Retain as conversation or generated material, not authority or candidate by itself.

Action class	Data class	Autonomy level	Authorization	Trace/Recovery
Update known material	Scoped module material	Bounded mutation	Required when provenance, truth metadata, scope, or sensitivity changes	Versioned KnowledgeRecord with correction and rollback; KnowledgeCandidate if boundary change is pending.
Invoke internal action	Small task-scoped non-sensitive input	Bounded internal action	Policy allow required	Requested action, input class, result, policy outcome, and recovery expectation recorded.
Contact remote service	Minimized and classified transfer set	External action	Explicit policy authorization	Transfer record, provider, purpose, and retention assumption recorded.
Modify credentials, identity, legal, financial, destructive, or executable state	Sensitive or critical data	Critical action	Strong human authorization	Block unless recovery or explicit irreversible authorization exists.

Knowledge-boundary updates, routing mutation, workflow mutation, and learning lifecycle behavior follow the same authority rule: a pending knowledge-boundary change is a **KnowledgeCandidate**; an evaluated behavior improvement is a **LearningCandidate**; neither becomes active merely by being produced.

2.4 Learning Lifecycle

Learning follows a controlled lifecycle: observation, evaluation, **LearningCandidate** creation, evidence collection, evaluation, promotion, active monitoring, and rollback. Knowledge handling remains distinct: a stored **KnowledgeRecord** is not learning, and a pending change to its owning knowledge boundary is a **KnowledgeCandidate**. Routing preferences, workflow improvements, and inference-selection improvements remain inactive unless the appropriate learning promotion rule validates them for a defined scope.

Threat Model

3.1 Threat Taxonomy

Threat	MADRE exposure	Primary control	Verification surface
Source as instruction	Untrusted text attempts to alter policy, action execution, or knowledge handling by being interpreted as an instruction.	Source classification and instruction/data separation.	Adversarial context scenario.
Sensitive disclosure	Private context leaks to a model, action, or remote service.	Data classification, minimization, and remote-transfer authorization.	Disclosure review.
Excessive agency	generated material causes over-broad action.	Autonomy levels and action-class policy.	Policy matrix review.
Unsafe action execution	Action invocation produces side effects outside task scope.	Policy-gated internal action boundary; remote, host-command, filesystem, and mutation actions require stronger authority.	Allowed-action and blocked-action scenarios.
Incorrect truth authority for known material	A KnowledgeRecord is used as factual authority despite unsuitable truthLevel or truthAuthority.	Required knowledge metadata, intended-use checks, correction, and reclassification path.	Non-authoritative knowledge-use and truth-correction scenario.
Inspection misuse	Local console is treated as authority or mutation path.	Read-first inspection boundary and shared policy model.	Local inspection negative scenario.

Threat	MADRE exposure	Primary control	Verification surface
Supply-chain or provider dependency	Model, dependency, or service changes behavior or availability.	Replaceable model backend and dependency policy.	Inference substitution review.
Remote transfer risk	Local data is sent outside the user's domain without classification, minimization, authorization, or recorded trace.	Local-first default and remote-transfer policy boundary.	Remote-contact blocked scenario.
Unbounded consumption	Delayed work consumes excessive compute, storage, or cost.	Resource budgets, queue limits, and status inspection.	Resource benchmark.
Trace/recovery gaps	A result cannot be explained, corrected, invalidated, superseded, rolled back, or detached.	<code>RuntimeJournal</code> , append-only <code>RuntimeEvent</code> , and recovery semantics.	Recovery trace scenario.
Inference output as false authority	Fluent generated material is treated as truth, policy, knowledge, behavior, routing, authority, or a candidate by itself.	Explicit generated-material handling and <code>PolicyDecision</code> boundary.	Output non-authority/non-candidate scenario.

Threat Catalogue

Threat family	Primary boundary	Initial response
Source as instruction	Context bundle construction	Separate source material from instruction and policy.
Sensitive disclosure	Data classification and remote transfer	Minimize, redact, authorize, and record disclosures.
Unsafe action execution	Action and workflow contracts	Require action-class policy, bounded internal-action scope, and recovery expectation.
Known material used under incorrect truth metadata	KnowledgeRecord use	Enforce provenance, <code>truthLevel</code> , <code>truthAuthority</code> , intended use, correction, and reclassification.
Inspection bypass	Local console and status surfaces	Read-first inspection with no privileged mutation path.
Generated-material false authority	generated material boundary	Treat generated material as produced material, not authority or candidate by itself.
Provider dependency	Local Model Runtime Boundary	model backend substitution and local-first default.
Unbounded consumption	Scheduler and resource controls	Queue, compute, storage, and cost limits.